

AGRIAI 2.0

INTELLIGENT FARMING ASSISTANT FOR GHANA

Volume VI. Sequence Poster Pack

Read these diagrams top to bottom. Each box is a real file or vendor, no abstract actors

Prepared and authored solely by Thaddeus Nii Teiko Tagoe

Computer Science, University of Ghana

Founder, designer, and sole engineer of AgriAI

Repository: ThaddeusHackz/agriai-v2

Document date: 18 August 2026

Version: 2.0.0 | Classification: Internal forensic architecture pack

This dossier is a forensic engineering record. It shows how AgriAI was built, which source files make the AI work, why answers still arrive when a vendor is down, and how the public and admin APIs stay reliable. Every claim is verified against the live codebase on 18 August 2026. No em dash character is used. The sole creator of every line of product code and of this dossier is Thaddeus Nii Teiko Tagoe.

Forensic scan executed on branch arena/01a0159e-agriai-v2, commit 9914572, Next.js 16.2.10, React 19.2.4, TypeScript 5, Tailwind CSS 4. Evidence tables in this pack list real file paths, real line counts, and real provider contracts. Reading alone will not make the system clear, the technical wiring will.

How to read the posters

Time flows downward. A line of the form A -> B : message means A calls B synchronously or over fetch. Alt blocks are fallbacks that happen when the first call is missing or throws. A bar like [SSE delta] means a Server Sent Event. These are study posters you can redraw on paper. They are not UML for a code generator, they are forensic memory aids.

Poster 1. Chat answer (the main brain, the most tested sequence)

```
Farmer typed in Twi, picked Expert, toggled webSearch on
-> components/Chat.tsx : collect message, language tw, mode expert, webSearch true, history, sessionId
-> fetch POST /api/chat/route.ts : Content-Type application/json { message, language tw, mode expert, webSearch, history, sessionId }
app/api/chat/route.ts -> lib/db.ts : trackQuestion(message), trackMessage()
alt webSearch == true
  route.ts -> lib/search.ts : searchWeb(message + " Ghana 2026")
  lib/search.ts -> try Tavily POST https://api.tavily.com/search advanced 6 results
  on missing key or Tavily error -> Wikipedia OpenSearch + DuckDuckGo Instant Answer
  lib/search.ts -> lib/search.ts contextBlock() : build numbered [1] [2] source block
else webSearch false, no search, sources = []
```

```

    route.ts -> getDB().settings.chat : pick systemPrompt or expertPrompt or agentPrompt by mode
    route.ts -> lib/languages.ts : languageInstruction("tw") : Reply in Twi ... keep English terms in parentheses
    route.ts : system = promptBase + languageInstruction + "You are AgriAI" + CURRENT WEB SEARCH RESULTS block
    alt GEMINI_API_KEY present (lib/env.ts firstEnv finds it)
    route.ts -> lib/ai.ts : geminiGenerateStream({ model = settings.chat.model, contents = last 8 turns + new message,
        systemInstruction = system, temperature, maxOutputTokens, signal request.signal, onDelta })
    lib/ai.ts -> Google Gemini : for each model in uniqueModels(preferred) over CHAT_MODELS 7 entries
        config includes thinkingConfig thinkingBudget 0 for 2.5 models
        client.models.generateContentStream, iterate async, chunk.text -> onDelta(delta) -> route.ts send("delta")
    route.ts -> Farmer browser : SSE event: delta data { text: delta } (rendered as it arrives)
        on success fullText = result.text, provider = "gemini"
    else or Gemini threw or empty ([chat] Gemini stream failed)
    route.ts -> lib/cloudflare.ts : cloudflareChat(cfMessages maxTokens temperature signal)
    lib/cloudflare.ts -> https://api.cloudflare.com/client/v4/accounts/{id}/ai/run/@cf/meta/llama-3.3-70b-instruct-fp8-fast
    Headers: Authorization Bearer CLOUDFLARE_API_KEY, body { messages last 16, max_tokens, temperature }
    on success fullText = response, provider = "cloudflare"
    else both providers empty or missing
    route.ts -> lib/ai.ts : localAnswer(message, "tw", "expert")
    lib/ai.ts -> getDB().knowledge scan keywords, then FALLBACK_ANSWERS regex, then generic card
        provider = "local", demo = true, sources = [] (no fake citations)
    route.ts -> lib/db.ts : mutate(push user turn + assistant turn into chats[], trackChat if new, updatedAt now)
    route.ts -> Farmer browser : SSE event: done data { text: fullText, demo, sources [{title url}], searchDemo, provider }
    <- Chat.tsx : render markdown via react-markdown + remark-gfm, show source chips [1] [2], badge provider and demo

```

Poster 1. Copy this to understand why answers still arrive when Gemini is down.

Poster 2. Sick leaf photo (vision, temperature 0.2)

```

Farmer on DiseaseDetector section
-> DiseaseDetector.tsx : user picks file via input file accept image/*
    FileReader readAsDataURL -> preview img src = data:image/jpeg;base64,...
-> fetch POST /api/vision : JSON { image: data URL, language: "en" } [approx 1 to 4 MB]
    app/api/vision/route.ts : check typeof image string and length >= 100 else 400
    check length <= 4.5 MB else 413 Image too large
    check language regex en|tw|ga|ee|ha|fr else "en"
    normalize: if not startsWith("data:") then "data:image/jpeg;base64," + image
-> lib/ai.ts : detectCropDisease(imageInput, language)
    parseImageData: regex ^data:([^;]+);base64,(.*)$ else fallback mime image/jpeg
    prompt = You are a crop disease expert for Ghana ... Return STRICT JSON { detected, confidence, description, treatment } temp 0.2
    alt no getGemini() (key missing)
        return FALLBACK_DISEASE { detected Unable to analyze (offline demo mode), confidence 0, demo true }
    else
        for model in uniqueModels(settings.chat.visionModel) over CHAT_MODELS
            client.models.generateContent({ model, contents [{ role user, parts [{ text prompt }, { inlineData { mimeType data } }] } ] },
                config { temperature 0.2 maxOutputTokens 1024 thinkingBudget 0 if 2.5 })

```

```

        raw = response.text, jsonMatch = raw.match(/\s\S*\s/), JSON.parse, return clamp
ed confidence and treatment slice 0..6
        on all throw return FALLBACK with detected Vision analysis failed and lastErr
        <- JSON { detected, confidence 0..100, description, treatment [], demo } status 200 or 500
Analysis failed
        <- DiseaseDetector.tsx : show detected name, confidence bar, description paragraph, treatment l
ist, demo badge yellow

```

Poster 3. Crop Visualizer image from text (studio, 37 word gate)

```

Farmer on CropStudio section
-> CropStudio.tsx : user types "healthy maize field at sunrise in Ghana" (must contain a farm w
ord)
-> fetch POST /api/generate : JSON { prompt: string trimmed 0..500 }
    app/api/generate/route.ts : if not cloudflareConfigured and not geminiConfigured -> 503 ne
eds keys
    if prompt empty -> 400 Describe what you want to visualize
    gate: low = prompt.toLowerCase(), SAFE_SUBJECTS = [maize corn cocoa cassava yam plantain r
ice tomato pepper ...] 37 entries
    relevant = SAFE_SUBJECTS.some(s => low.includes(s)) else 400 Please describe a crop
or farm scene
    fullPrompt = `${prompt}, photorealistic agricultural photography, lush healthy crops, gold
en hour lighting, high detail`
    alt cloudflareConfigured true
    -> lib/cloudflare.ts : cloudflareImage(fullPrompt, { steps: 4 })
    POST https://api.cloudflare.com/client/v4/accounts/{id}/ai/run/@cf/black-forest-labs/
flux-1-schnell
    body { prompt: fullPrompt, num_steps: 4 } -> result.image base64 -> return data:image
/png;base64, + b64
    <- { ok true, image: data URL, provider: "cloudflare" } 200
    else or Cloudflare threw ([generate] Cloudflare image failed)
    alt geminiConfigured true
    -> lib/ai.ts : geminiImage(fullPrompt)
    for model in IMAGE_MODELS 3 entries: gemini-2.5-flash-image, gemini-2.0-flash-previ
ew-image-generation, gemini-2.0-flash-exp-image-generation
    client.models.generateContent({ model, contents [{ role user parts [{ text fullProm
pt }]}], config { responseModalities [IMAGE,TEXT] temperature 0.8 } })
    scan candidates[0].content.parts for inlineData.data -> return data:{mime};base64,
+ data
    <- { ok true, image: data URL, provider: "gemini" }
    else 502 Image generation failed
    <- CropStudio.tsx : setImage(data URL), show img, enable Download PNG button

```

Poster 4. Voice in and voice out (two uses of one Gemini key)

```

VOICE IN primary path (no quota)
Farmer mic button -> Chat.tsx : if window.SpeechRecognition or webkitSpeechRecognition exists
    start Recognition, interimResults false, lang speechHint from LANGUAGES (en-GH or fr-FR)
    onresult event.results[0][0].transcript -> insert into message input box
VOICE IN fallback path (quota)
Farmer mic button -> Chat.tsx : else no SpeechRecognition, start MediaRecorder audio/webm
    onstop Blob -> FormData field audio File -> fetch POST /api/transcribe multipart
    app/api/transcribe/route.ts : getGemini or 501 Voice input not configured
    check FormData audio is File else 400 No audio, size > 25 MB else 413
    Buffer.from(await audio.arrayBuffer()).toString("base64")
    -> lib/ai.ts : geminiGenerateText({ model gemini-2.5-flash temp 0 maxOutputTokens 1024 conten
ts [{ role user parts [{ text Transcribe verbatim }, { inlineData { mimeType data } }] }] })
    <- JSON { text: transcribed string } 200 else 500 Transcription failed
    Chat.tsx : text into input, farmer presses Send as normal chat
VOICE OUT
Farmer taps speaker on an answer -> Chat.tsx : fetch POST /api/tts { text: answer sliced 0..1500,
language, speed 1.0 }

```

```

app/api/tts/route.ts : elevenLabsApiKey or 501 Voice output not configured
voiceId = VOICES[language] or VOICES.default : en->21m00Tcm4TlvDq8ikWAM Rachel, fr same, default
t AZnzlk1XvdvUeBnXmllid multilingual
-> POST https://api.elevenlabs.io/v1/text-to-speech/{voiceId} Headers Accept audio/mpeg xi-api-
key key
  body { text clean, model_id eleven_multilingual_v2, voice_settings { stability 0.5 similarit
y_boost 0.75 style 0.0 speed clamped 0.5..2 } }
  <- status 200 bytes audio/mpeg with Cache-Control public max-age 3600 else 502 Voice generation
failed
Chat.tsx : new Audio(URL.createObjectURL(blob)) play(), show stop button

```

Poster 5. Admin login and prompt edit (the control room)

```

Staff opens /admin -> redirect to /admin/login if no cookie agriai_session
Staff types email password remember boolean -> POST /api/admin/login
app/api/admin/login/route.ts : ip = clientIp(req) from x-forwarded-for first hop or x-real-ip
if isRateLimited(ip) -> 429 Too many failed attempts
try JSON parse else 400 Invalid request
email trimmed lower, password required else 400 Email and password are required
findUserByEmail(email) or 401, verifyPassword(password, hash) via bcrypt compare else recordFail
edAttempt 401 Invalid email or password
clearAttempts(ip)
token = startSession(email, ip, user-agent) : 32 random hex, push to sessions with expiresAt no
w + 7 days, prune expired
mutate set lastLogin now, NextResponse { ok true user publicUser } withSessionCookie token reme
mber ? 7 days : 8 hours
Browser stores httpOnly cookie, next fetch to /admin includes it automatically
Staff in AdminShell tab AITab edits fields
components/admin/AITab.tsx : model dropdown, temperature slider 0..1, maxTokens 256..2048, syst
emPrompt expertPrompt agentPrompt textarea, visionModel
-> components/admin/api.ts : PUT /api/admin/settings { chat { model temperature maxTokens syste
mPrompt expertPrompt agentPrompt visionModel } plus site fields }
app/api/admin/settings/route.ts : getSessionUser(req) or 401 Unauthorized
validate JSON strings lengths, mutate((db)=> db.settings = { ...db.settings, ...body, chat: {
...db.settings.chat, ...body.chat }, ... } )
  <- { ok true settings }
Next farmer POST /api/chat reads getDB().settings.chat immediately, no rebuild, no redeploy

```

Poster 6. Boot and memory survival (why Render wipes do not lose chats)

```

Next.js process start (first request or deploy scale up)
-> instrumentation.ts : export async function register() { if !postgresConfigured return; ok =
await initPostgres(); if !ok return; await hydrateFromPostgres() => postgresLoad() }
lib/pg-store.ts initPostgres : Pool connectionString DATABASE_URL ssl rejectUnauthorized fal
se when sslmode require, query CREATE TABLE IF NOT EXISTS agriai_state (id PK, doc JSONB, updated
_at timestamptz)
lib/db.ts hydrateFromPostgres : if fs.existsSync(data/db.json) return (disk wins)
  doc = await postgresLoad() SELECT doc FROM agriai_state WHERE
id=1
  if doc { cache = doc; persist(doc) console.log hydrated }
else no DATABASE_URL or file exists
-> lib/db.ts : load() { if exists parse JSON, merge missing keys with defaultDB, return; else
defaultDB() }
  defaultDB : admin user bcrypt hashSync(ADMIN_PASSWORD 10), SEED_PRICES 12 entries each id
uid prc, SEED_KNOWLEDGE 8 entries each id uid kno updatedAt now, analytics firstSeen now, setting
s default with 2026 aurora colors
  persist(db) { mkdir data recursive, writeFileSync db.json.tmp JSON 2 spaces, renameSync to
db.json atomic, schedulePostgresSave(db) }
Request write path (any mutate)
mutate((db)=> { db.chats.push(...); db.visits... } ) -> persist -> rename atomic -> scheduleP

```

```
ostgresSave debounce 1500 ms
  schedulePostgresSave { pendingDoc = db; if flushTimer return; setTimeout flushPostgres 1500 }
  flushPostgres { Pool query INSERT agriai_state id 1 doc jsonb ON CONFLICT UPDATE doc updated_
at now() }
```

Poster 7. Full provider ladder on one page (the poster you memorize)

```
PROVIDER LADDER (read top to bottom, first non empty wins)
CHAT text      : Gemini stream (7 models waterfall thinkingBudget 0) -> Cloudflare Llama 3.3 70B
30 sec -> localAnswer db.knowledge plus FALLBACK_ANSWERS regex
VISION photo   : Gemini multimodal STRICT JSON detected confidence description treatment temp 0.2
-> FALLBACK_DISEASE demo card (offline text + MoFA advice)
IMAGE prompt   : Cloudflare Flux 1 schnell 4 steps -> Gemini image 3 models responseModalities IM
AGE TEXT -> 502 error
SEARCH query   : Tavily advanced 6 results include_answer true -> Wikipedia OpenSearch 4 + DuckDu
ckGo Instant Answer -> empty sources with demo true
WEATHER city   : OpenWeatherMap 2.5 weather + forecast 5 days accra kumasi tamale takoradi cape c
oast -> Open-Meteo Africa Accra 5 days -> seedForecast deterministic pseudo random adviceFor
VOICE in       : Browser SpeechRecognition en-GH fr-FR hint -> Gemini transcribe inline audio gem
ini-2.5-flash verbatim
VOICE out      : ElevenLabs eleven_multilingual_v2 Rachel for en fr default multilingual -> disab
le speak on 501
STATE write    : db.json atomic tmp rename -> PostgreSQL agriai_state JSONB debounced 1500 ms
STATE read    : instrumentation hydration if db.json missing and DATABASE_URL set, else load or
defaultDB seed
AUTH          : bcrypt hash 10, session token 32 hex, cookie httpOnly lax secure, rate limit 5 p
er 15 min Map keyed by x-forwarded-for
AUTHZ         : getSessionUser cookie -> admin or editor role -> guard /api/admin/* and prices P
OST DELETE
```

Poster 8. The whole system on one page (from pixel to Postgres)

```
Farmer browser (phone) ---- fetch /api/* only ----> Next.js 16 App Router nodejs runtime
components: AnnouncementBar Nav Hero Chat DiseaseDetector CropStudio WeatherSection MarketPrice
s FAQ Newsletter Footer Founder
Chat: SSE client delta done markdown sources language tw ga ee ha fr mode standard expert agent
lib/env.ts: firstEnv trim quote strip placeholder reject providerStatus
lib/ai.ts: getGemini cached, CHAT_MODELS 7, IMAGE_MODELS 3, geminiGenerateText geminiGenerateSt
ream geminiImage detectCropDisease localAnswer FALLBACK
lib/cloudflare.ts: cloudflareChat Llama3.3 70B, cloudflareImage Flux, POST bearer 30 sec
lib/search.ts: searchWeb Tavily plus keyless Wikipedia DuckDuckGo contextBlock numbered sources
lib/openweather.ts: openweatherForecast weather+forecast aggregateDaily, openMeteo fallback, se
edForecast
lib/db.ts: getDB cache, defaultDB seeds 12 prices 8 knowledge admin bcrypt, load merge, persist
atomic, mutate, trackVisit trackQuestion trackChat trackMessage
lib/pg-store.ts: Pool 3 idle 30s connect 8s, initPostgres, postgresLoad select, schedulePostgre
sSave debounce 1500 upsert
lib/auth.ts: hash verify, isRateLimited recordFailedAttempt clearAttempts, createSessionToken 3
2 hex startSession endSession getSessionUser withSessionCookie clearSessionCookie clientIp public
User
lib/languages.ts: LANGUAGES 6 map languageInstruction
instrumentation.ts: register hydrates if missing file and Postgres
Next.js build output .next + public -> Render web service or Docker node:22-alpine port 3000 he
alth /
Data lives at data/db.json gitignored mirrored to Postgres agriai_state id 1
Admin at /admin login rate limited 5 per 15 min cookie agriai_session -> Tabs Appearance AI Con
tent Prices Knowledge Messages Users Feedback Subscribers Settings
Vendors outside: Google Gemini, Cloudflare Workers AI, Tavily, OpenWeather, Open-Meteo, ElevenL
abs, Unsplash, Wikipedia, DuckDuckGo
All arrows owned by Thaddeus Nii Teiko Tagoe, sole creator
```

Why these posters matter more than paragraphs

Paragraphs explain. Posters let a technical reviewer see the full wiring in 10 seconds and then immediately open the named file. If you can redraw Poster 1 and Poster 7 from memory you understand how AgriAI intelligence was made to work: a ladder of real APIs ordered by speed and cost, Ghana prompts that change voice without retraining, search citations that ground without a vector database, and a local brain that guarantees the product never breaks. Every box in every poster was authored in TypeScript by Thaddeus Nii Teiko Tagoe, sole creator of AgriAI 2.0 and of this forensic pack built on 18 August 2026.